

Lab 8. Polymorphism & Abstraction

Writing Good Programs

The only way to learn programming is program, program and program. Learning programming is like learning cycling, swimming or any other sports. You can't learn by watching or reading books. Start to program immediately. On the other hands, to improve your programming, you need to read many books and study how the masters program.

It is easy to write programs that work. It is much harder to write programs that not only work but also easy to maintain and understood by others – I call these good programs. In the real world, writing program is not meaningful. You have to write good programs, so that others can understand and maintain your programs.

Pay particular attention to:

1. Coding style:

- Read Java code convention: "Google Java Style Guide" or "Java Code Conventions - Oracle".
- Follow the Java Naming Conventions for variables, methods, and classes STRICTLY. Use CamelCase for names. Variable and method names begin with lowercase, while class names begin with uppercase. Use nouns for variables (e.g., radius) and class names (e.g., Circle). Use verbs for methods (e.g., getArea(), isEmpty()).
- **Use Meaningful Names:** Do not use names like a, b, c, d, x, x1, x2, and x1688 - they are meaningless. Avoid single-alphabet names like i, j, k. They are easy to type, but usually meaningless. Use single-alphabet names only when their meaning is clear, e.g., x, y, z for co-ordinates and i for array index. Use meaningful names like row and col (instead of x and y, i and j, x1 and x2), numStudents (not n), maxGrade, size (not n), and upperbound (not n again). Differentiate between singular and plural nouns (e.g., use books for an array of books, and book for each item).
- Use consistent indentation and coding style. Many IDEs (such as Eclipse / NetBeans) can re-format your source codes with a single click.

2. Program Documentation: Comment! Comment! and more Comment to explain your code to other people and to yourself three days later.

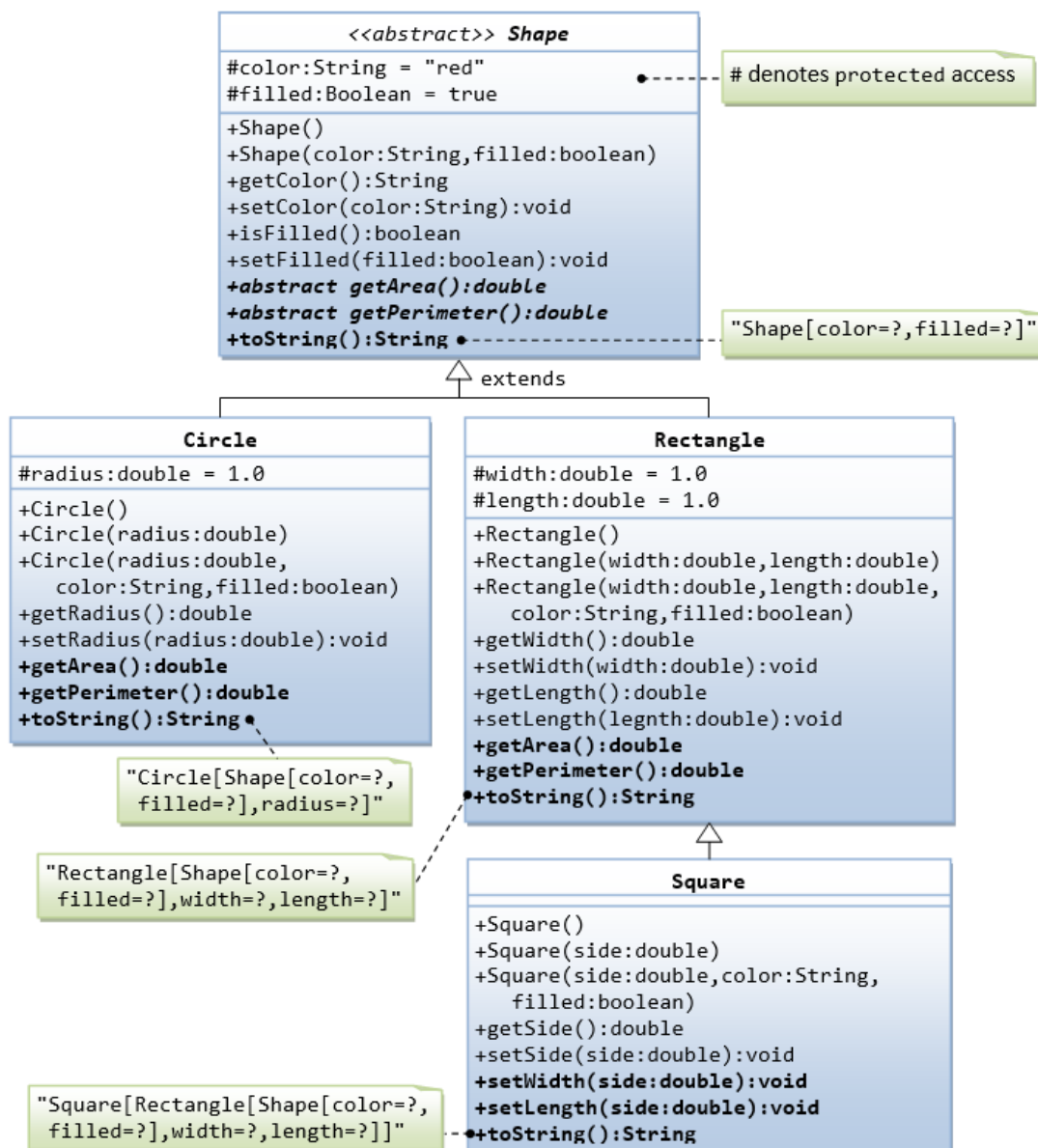
3. The problems in this tutorial are certainly NOT challenging. There are tens of thousands of challenging problems available – used in training for various programming contests (such as International Collegiate Programming Contest (ICPC), International Olympiad in Informatics (IOI)).

1 Exercises on Polymorphism, Abstract Classes and Interfaces

1.1 Abstract Superclass Shape and Its Concrete Subclasses

Rewrite the superclass Shape and its subclasses Circle, Rectangle and Square, as shown in the class diagram.

Shape is an abstract class containing 2 abstract methods: *getArea()* and *getPerimeter()*, where its concrete subclasses must provide its implementation. All instance variables shall have protected access, i.e., accessible by its subclasses and classes in the same package. Mark all the overridden methods with annotation `@Override`.



In this exercise, Shape shall be defined as an abstract class, which contains:

- Two protected instance variables `color(String)` and `filled(boolean)`. The protected variables can be accessed by its subclasses and classes in the same package. They are denoted with a '#' sign in the class diagram.
- Getter and setter for all the instance variables, and `toString()`.
- Two abstract methods `getArea()` and `getPerimeter()` (shown in italics in the class diagram).

The subclasses Circle and Rectangle shall override the abstract methods `getArea()` and `getPerimeter()` and provide the proper implementation. They also override the `toString()`.

Write a test class to test these statements involving polymorphism and explain the outputs. Some statements may trigger compilation errors. Explain the errors, if any.



```

1 Shape shape1 = new Circle(5.5, "red", false); // Upcast Circle to Shape
  System.out.println(shape1);                  // which version?
3 System.out.println(shape1.getArea());          // which version?
  System.out.println(shape1.getPerimeter());    // which version?
5 System.out.println(shape1.getColor());
  System.out.println(shape1.isFilled());
7 System.out.println(shape1.getRadius());

9 Circle circle1 = (Circle)shape1;              // Downcast back to Circle
  System.out.println(circle1);
11 System.out.println(circle1.getArea());
  System.out.println(circle1.getPerimeter());
13 System.out.println(circle1.getColor());
  System.out.println(circle1.isFilled());
15 System.out.println(circle1.getRadius());

17 Shape shape2 = new Shape();

19 Shape shape3 = new Rectangle(1.0, 2.0, "red", false); // Upcast
  System.out.println(shape3);
21 System.out.println(shape3.getArea());
  System.out.println(shape3.getPerimeter());
23 System.out.println(shape3.getColor());
  System.out.println(shape3.getLength());
25
  Rectangle rectangle1 = (Rectangle)shape3; // downcast
27 System.out.println(rectangle1);
  System.out.println(rectangle1.getArea());
29 System.out.println(rectangle1.getColor());
  System.out.println(rectangle1.getLength());
31
  Shape shape4 = new Square(6.6); // Upcast
33 System.out.println(shape4);
  System.out.println(shape4.getArea());
35 System.out.println(shape4.getColor());

```



```

System.out.println(shape4.getSide());
37
// Take note that we downcast Shape shape4 to Rectangle,
39 // which is a superclass of Square, instead of Square
Rectangle rectangle2 = (Rectangle)shape4;
41 System.out.println(rectangle2);
System.out.println(rectangle2.getArea());
43 System.out.println(rectangle2.getColor());
System.out.println(rectangle2.getSide());
45 System.out.println(rectangle2.getLength());

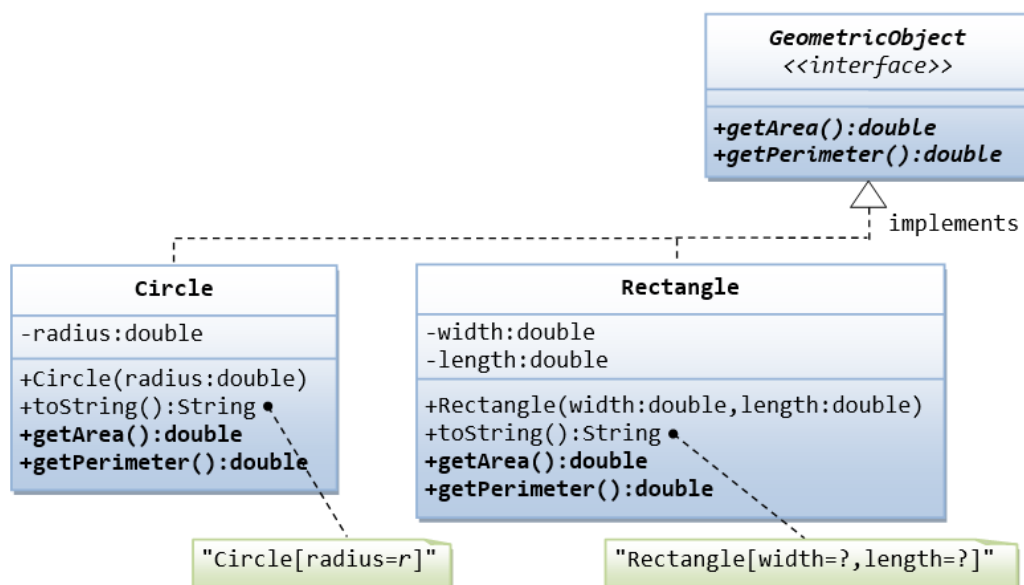
47 // Downcast Rectangle rectangle2 to Square
Square square1 = (Square)rectangle2;
49 System.out.println(square1);
System.out.println(square1.getArea());
51 System.out.println(square1.getColor());
System.out.println(square1.getSide());
53 System.out.println(square1.getLength());

```

What is the usage of the abstract method and abstract class?

1.2 GeometricObject Interface and its Implementation Classes Circle and Rectangle

Write an interface called `GeometricObject`, which contains 2 abstract methods: `getArea()` and `getPerimeter()`, as shown in the class diagram. Also write an implementation class called `Circle`. Mark all the overridden methods with annotation `@Override`.

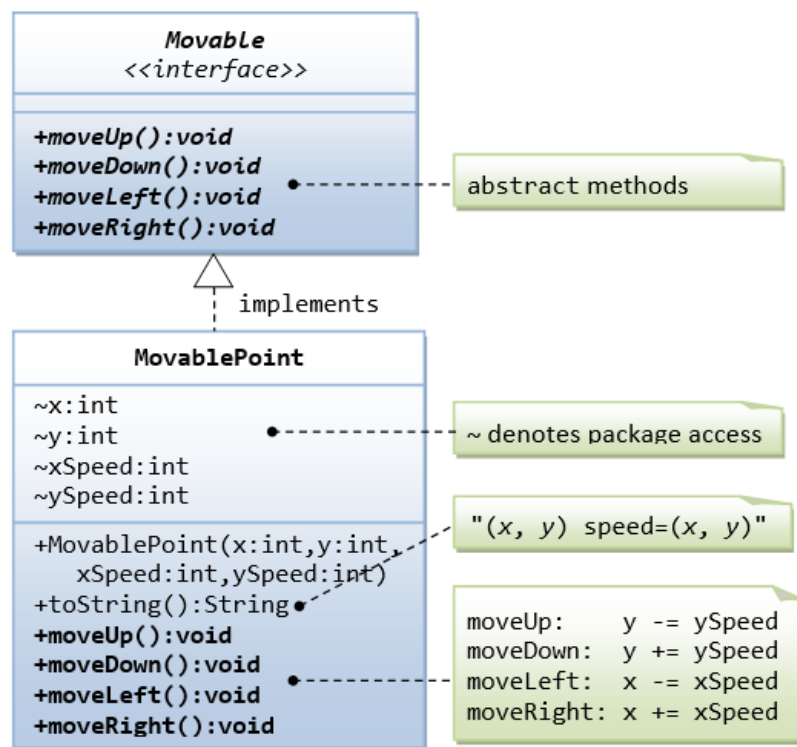


1.3 Movable Interface and its Implementation MovablePoint Class

Write an interface called Movable, which contains 4 abstract methods:

- *moveUp()*
- *moveDown()*
- *moveLeft()*
- *moveRight()*

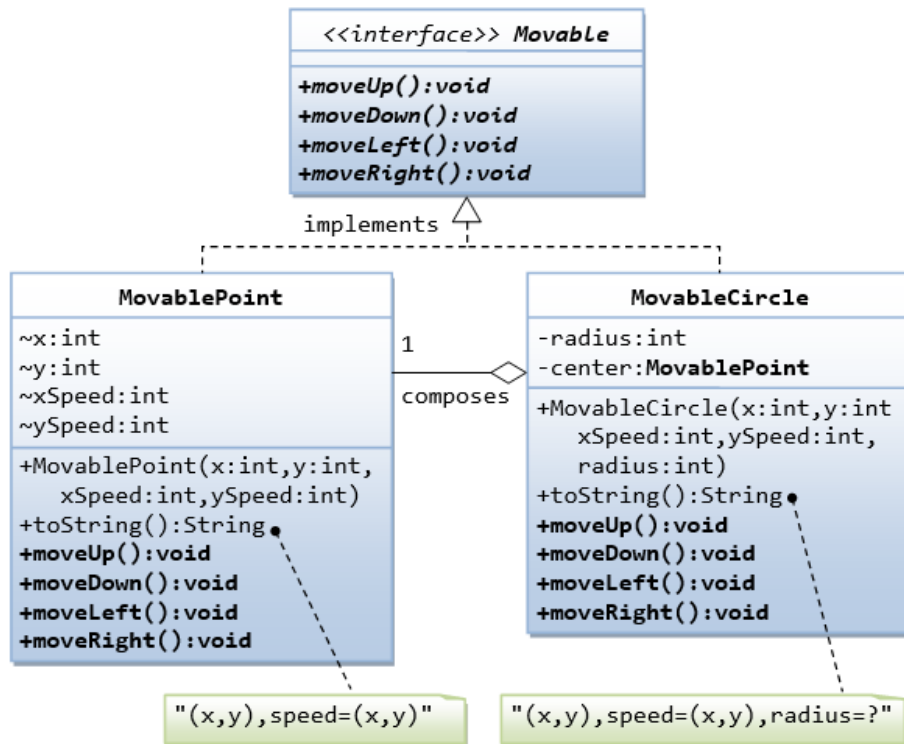
as shown in the class diagram.



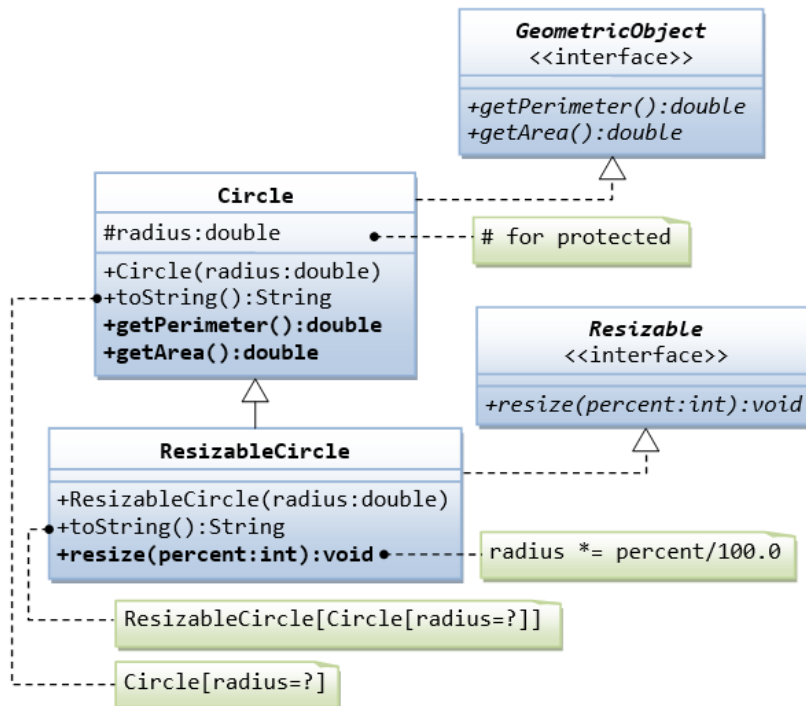
Also write an implementation class called MovablePoint. Mark all the overridden methods with annotation `@Override`.

1.4 Movable Interface and its Implementation Classes MovablePoint and MovableCircle

Write an interface called Movable, which contains 4 abstract methods *moveUp()*, *moveDown()*, *moveLeft()* and *moveRight()*, as shown in the class diagram. Also write the implementation classes called MovablePoint and MovableCircle. Mark all the overridden methods with annotation `@Override`.



1.5 Interfaces Resizable and GeometricObject



1. Write the interface called `GeometricObject`, which declares two abstract methods: `getParameter()` and `getArea()`, as specified in the class diagram.

Hints:



```
1 public interface GeometricObject {  
    public double getPerimeter();  
3     .....  
    }
```

2. Write the implementation class `Circle`, with a protected variable `radius`, which implements the interface `GeometricObject`.

Hints:



```
1 public class Circle implements GeometricObject {  
    // Private variable  
3     .....  
  
5    // Constructor  
    .....  
7  
    // Implement methods defined in the interface GeometricObject  
9    @Override  
    public double getPerimeter() { ..... }  
11  
    .....  
13 }
```

3. Write a test program called `TestCircle` to test the methods defined in `Circle`.
4. The class `ResizableCircle` is defined as a subclass of the class `Circle`, which also implements an interface called `Resizable`, as shown in class diagram. The interface `Resizable` declares an abstract method `resize()`, which modifies the dimension (such as radius) by the given percentage. Write the interface `Resizable` and the class `ResizableCircle`.

Hints:



```
public interface Resizable {  
2    public double resize(...);  
    }
```



```

public class ResizableCircle extends Circle implements Resizable {
2
    // Constructor
4    public ResizableCircle(double radius) {
        super (...);
6    }

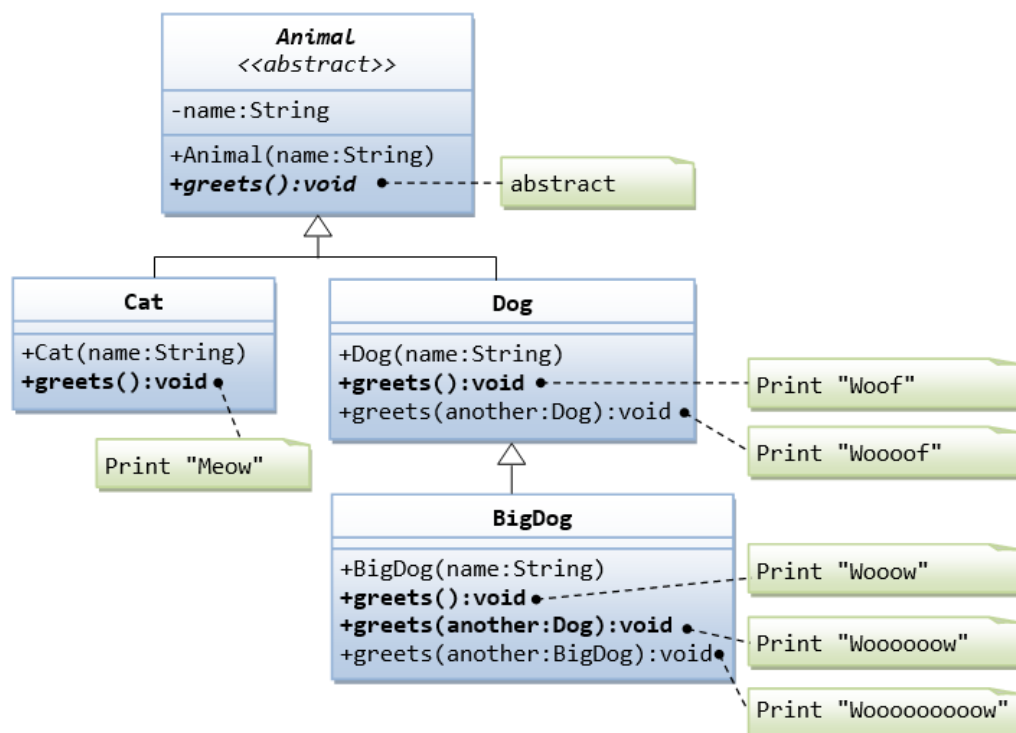
8    // Implement methods defined in the interface Resizable
    @Override
10    public double resize(int percent) { ..... }
}

```

- Write a test program called TestResizableCircle to test the methods defined in ResizableCircle.

1.6 Abstract Superclass Animal and its Implementation Subclasses

Write the codes for all the classes shown in the class diagram. Mark all the overridden methods with annotation @Override.



1.7 Another View of Abstract Superclass Animal and its Implementation Subclasses

Examine the following codes and draw the class diagram.



```
public abstract class Animal {  
2  abstract public void greeting();  
}
```



```
1 public class Cat extends Animal {  
    @Override  
3  public void greeting() {  
    System.out.println("Meow!");  
5  }  
}
```



```
public class Dog extends Animal {  
2  @Override  
    public void greeting() {  
4      System.out.println("Woof!");  
    }  
6  
    public void greeting(Dog another) {  
8      System.out.println("Wooooooooooof!");  
    }  
10 }
```



```
public class BigDog extends Dog {  
2  @Override  
    public void greeting() {  
4      System.out.println("Woow!");  
    }  
6  
    @Override  
8  public void greeting(Dog another) {  
    System.out.println("Woooooowwww!");  
10 }  
}
```

Explain the outputs (or error) for the following test program.



```

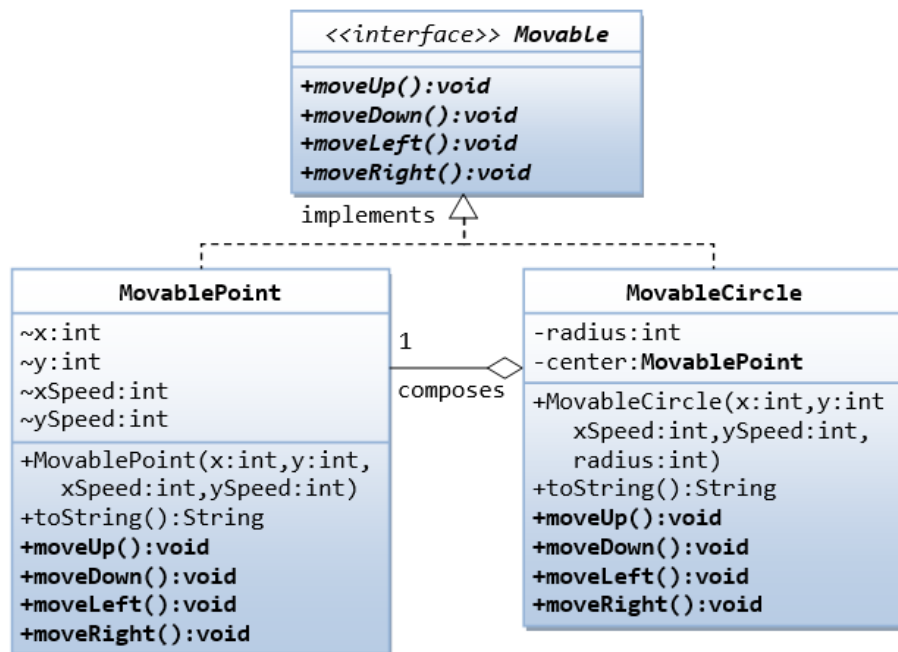
1 public class TestAnimal {
    public static void main(String[] args) {
2         // Using the subclasses
3         Cat cat1 = new Cat();
4         cat1.greeting();
5         Dog dog1 = new Dog();
6         dog1.greeting();
7         BigDog bigDog1 = new BigDog();
8         bigDog1.greeting();
9
10        // Using Polymorphism
11        Animal animal1 = new Cat();
12        animal1.greeting();
13        Animal animal2 = new Dog();
14        animal2.greeting();
15        Animal animal3 = new BigDog();
16        animal3.greeting();
17        Animal animal4 = new Animal();
18
19        // Downcast
20        Dog dog2 = (Dog) animal2;
21        BigDog bigDog2 = (BigDog) animal3;
22        Dog dog3 = (Dog) animal3;
23        Cat cat2 = (Cat) animal2;
24        dog2.greeting(dog3);
25        dog3.greeting(dog2);
26        dog2.greeting(bigDog2);
27        bigDog2.greeting(dog2);
28        bigDog2.greeting(bigDog1);
29    }
30 }
31 }

```

1.8 Interface Movable and its implementation subclasses MovablePoint and MovableCircle

Suppose that we have a set of objects with some common behaviors: they could move up, down, left or right. The exact behaviors (such as how to move and how far to move) depend on the objects themselves. One common way to model these common behaviors is to define an interface called *Movable*, with abstract methods *moveUp()*, *moveDown()*, *moveLeft()* and *moveRight()*. The classes that implement the *Movable* interface will provide actual implementation to these abstract methods.

Let's write two concrete classes - *MovablePoint* and *MovableCircle* - that implement the *Movable* interface.



The code for the interface `Movable` is straight forward.



```

1 public interface Movable { // saved as "Movable.java"
    public void moveUp();
3     .....
    }
  
```

For the `MovablePoint` class, declare the instance variable `x`, `y`, `xSpeed` and `ySpeed` with package access as shown with `' '` in the class diagram (i.e., classes in the same package can access these variables directly). For the `MovableCircle` class, use a `MovablePoint` to represent its center (which contains four variable `x`, `y`, `xSpeed` and `ySpeed`). In other words, the `MovableCircle` composes a `MovablePoint`, and its radius.



```

    public class MovablePoint implements Movable { // saved as "MovablePoint.
        ↪ java"
2        // instance variables
        int x;
4        int y;
        int xSpeed;
6        int ySpeed; // package access

8        // Constructor
        public MovablePoint(int x, int y, int xSpeed, int ySpeed) {
10            this.x = x;
  
```



```

12 }
    .....
14
    // Implement abstract methods declared in the interface Movable
16 @Override
    public void moveUp() {
18     y -= ySpeed;    // y-axis pointing down for 2D graphics
    }
20     .....
    }

```



```

1 public class MovableCircle implements Movable { // saved as "
    ⇨ MovableCircle.java"
    // instance variables
3 private MovablePoint center;    // can use center.x, center.y directly
    // because they are package accessible
5 private int radius;

7 // Constructor
    public MovableCircle(int x, int y, int xSpeed, int ySpeed, int radius)
        ⇨ {
9     // Call the MovablePoint's constructor to allocate the center instance.
        center = new MovablePoint(x, y, xSpeed, ySpeed);
11     .....
    }
13     .....

15 // Implement abstract methods declared in the interface Movable
    @Override
17 public void moveUp() {
        center.y -= center.ySpeed;
19     }
    .....
21 }

```

Write a test program and try out these statements:



```

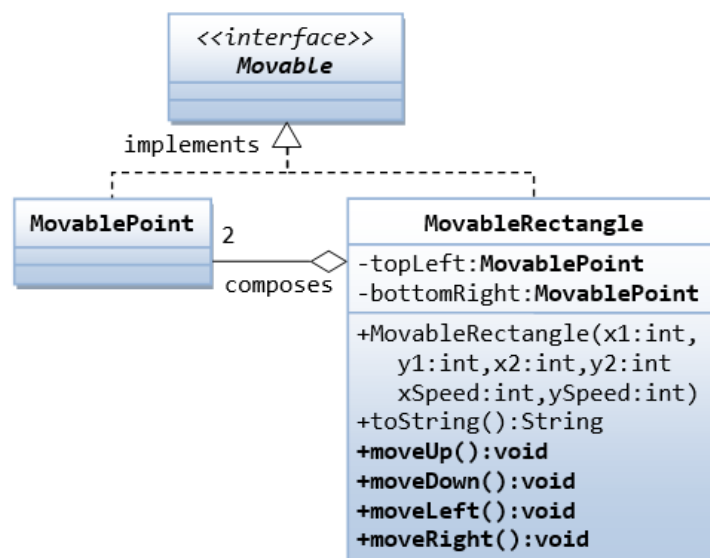
1 Movable m1 = new MovablePoint(5, 6, 10, 15);    // upcast
    System.out.println(m1);
3 m1.moveLeft();
    System.out.println(m1);
5
    Movable m2 = new MovableCircle(1, 2, 3, 4, 20); // upcast

```



```
7 System.out.println(m2);  
  m2.moveRight();  
9 System.out.println(m2);
```

Write a new class called `MovableRectangle`, which composes two `MovablePoints` (representing the top-left and bottom-right corners) and implementing the `Movable` Interface. Make sure that the two points has the same speed.



What is the difference between an interface and an abstract class?